

JavaScript Programming

A practical study reference with foundational concepts, examples, and common review questions.

1. Language Fundamentals

JavaScript runs in browsers and server runtimes such as Node.js. `let` and `const` are block-scoped; prefer `const` unless rebinding is required. Primitives include `string`, `number`, `boolean`, `undefined`, `symbol`, `bigint`, and `null`; objects, arrays, and functions are references.

Use `===` and `!==` to avoid most coercive equality surprises. Empty arrays and objects are truthy. `==` converts types. Closures retain access to lexical variables; arrow functions capture this from the surrounding scope.

2. Objects and Asynchronous Code

Objects store properties; arrays are ordered collections. Destructuring extracts values; spread syntax makes shallow copies, so nested objects may remain shared. Modules use `import` and `export` to state dependencies.

Promises represent future completion. `async` functions return Promises; `await` pauses that function, not the entire runtime. Handle rejection with `try/catch`. `Promise.all` suits independent tasks that must all succeed; `AbortController` can cancel fetch requests.

```
async function load(url) {  
  const response = await fetch(url);  
  if (!response.ok) throw new Error(String(response.status));  
  return response.json();  
}
```

3. Browser APIs and Security

The DOM represents the page. Use `querySelector` and event listeners to interact with it. Prefer `textContent` for plain text; assigning untrusted input to `innerHTML` can cause cross-site scripting. Validate data on the server even when the browser checks it.

Do not place API secrets in browser code. Web storage is accessible to same-origin scripts; cookies support controls such as `HttpOnly`, `Secure`, and `SameSite`. Use tests, linting, clear module boundaries, and explicit handling of asynchronous failures.

4. Common Questions

`null` is an explicit empty value; `undefined` often means absent or unassigned. A closure is a function plus access to its lexical environment. Event bubbling propagates an event from its target through ancestors. `await` unwraps a Promise result or throws its rejection in an `async` function.